

# Symbolic Techniques for Model Checking and State Space Exploration

Fabio Somenzi

University of Colorado at Boulder

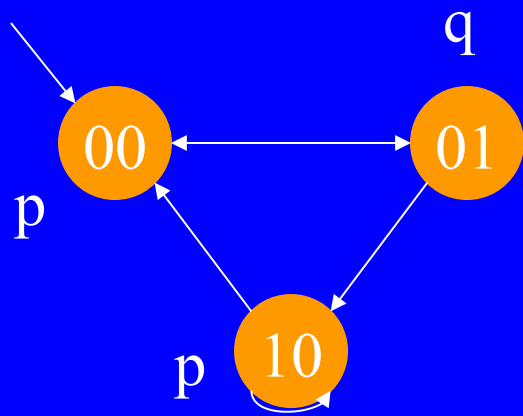
# Outline

- Characteristic functions and BDDs
  - Symbolic Model Checking
  - Image/preimage computation
  - Fixpoint computation
  - Cycle detection
  - Don't care conditions

# Symbolic Representation

- Each set  $S$  is represented by its characteristic function,  $\chi_S : S \rightarrow \{0,1\}$
- Algorithms avoid enumeration of individual members of the sets
- Potentially very concise:  $x_1 \vee x_{100}$  represents  $3 \cdot 2^{98}$  elements; however...
- For a given representation, most functions have exponential description: no free lunch
- BDDs often used for characteristic functions

# Symbolic Representation



- $T(x_1, x_0, y_1, y_0) =$   
 $\neg x_1 \wedge \neg x_0 \wedge \neg y_1 \wedge y_0 \vee$   
 $\neg x_1 \wedge x_0 \wedge \neg y_0 \vee$   
 $x_1 \wedge \neg x_0 \wedge \neg y_0$
- $S_0(x_1, x_0) = \neg x_1 \wedge \neg x_0$
- $p(x_1, x_0) = \neg x_0$
- $q(x_1, x_0) = \neg x_1 \wedge x_0$

# Binary Decision Diagrams

- BDDs are reduced decision trees
- For a given variable order BDDs are canonical
- Most interesting operations on BDDs are linear in the size of each operand
- BDDs use lots of memory
- Variable order may have large impact
- Plain, old BDDs still best for model checking [Yang et al., FMCAD'98]

# Operations on BDDs

The most important for model checking:

- Conjunction
- Quantification
- AndExists
- Minimization (Constrain, Restrict, ...)
- Approximation, Decomposition, ...

# Variable Ordering

- There may be an exponential gap in size between a good order and a bad one
- Dynamic variable ordering (DVO) often essential, but expensive
- Good orders found with DVO are often saved to speed up subsequent runs
- DVO is (almost) transparent to the application
- Sifting is the most popular approach to DVO
- MC issue: relative position of variable pairs

# Outline

- ✓ Characteristic functions and BDDs
- Symbolic Model Checking
  - Image/preimage computation
  - Fixpoint computation
  - Cycle detection
  - Don't care conditions

# CTL\* (Loosely Speaking)

- Formulae are made up of atomic propositions ( $p, q, \dots$ ) and operators
  - $\neg, \wedge$
  - $X\varphi$ : along this path  $\varphi$  holds in the next state
  - $\psi U \varphi$ : along this path  $\psi$  holds until  $\varphi$  holds
  - $E\varphi$ : there is a path along which  $\varphi$  holds
- Legal formulae say something about the states of the model
- Example:  $E p U (q \wedge \neg X p)$

# Abbreviations and Extensions

- $F \varphi : \text{true } U \varphi$
- $G \varphi : \neg F \neg \varphi$
- $A \varphi : \neg E \neg \varphi$
- Past time operators:
  - $X \leftrightarrow Y, U \leftrightarrow S, F \leftrightarrow P, G \leftrightarrow H$
  - Example:  $Y \varphi$  : along this path  $\varphi$  held at the previous state

# How to Model Check EF and EG

- The states satisfying EF and EG formulae are fixpoints of monotonic functions over  $2^S$ 
  - $EF \varphi = \varphi \vee EX EF \varphi$
  - $EG \varphi = \varphi \wedge EX EG \varphi$
- Specifically,  $EF \varphi$  is a least fixpoint and  $EG \varphi$  is a greatest fixpoint. This is written
  - $EF \varphi = \mu Z . \varphi \vee EX Z$
  - $EG \varphi = \nu Z . \varphi \wedge EX Z$

# Computing $EF_\varphi$ and $EG_\varphi$

```
Z =  $\emptyset$ ;  
 $\zeta$  = S;
```

```
While (Z  $\neq$   $\zeta$ ) {  
     $\zeta$  = Z;  
    Z =  $\varphi^\vee$  EX Z;  
}
```

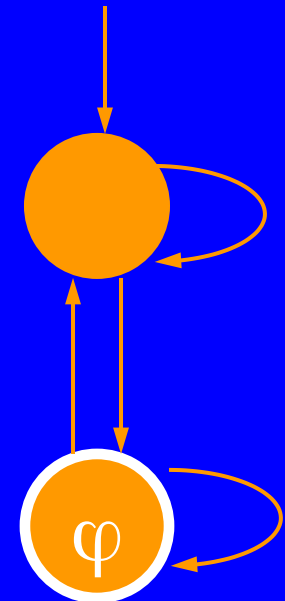
```
Z = S;  
 $\zeta$  =  $\emptyset$ ;
```

```
While (Z  $\neq$   $\zeta$ ) {  
     $\zeta$  = Z;  
    Z =  $\varphi^\wedge$  EX Z;  
}
```

# How to Model Check $\text{EGF}_\varphi$

- Translate into Büchi automaton
- Compose automaton with model
- Check composition for a fair path
- A fair path satisfies all acceptance conditions infinitely often
- Computation of fair paths

$$\forall Z. \bigwedge_i \text{EX} [\text{EZU} (Z \wedge c_i)]$$



# CTL\* Model Checking

- We need to be able to
- Translate formulae like  $\text{EGF}_\varphi$  into automata
- Compute fixpoints like
  - $\Box \mu Z. \varphi \vee \text{EX } Z$
  - $\Box \nu Z. \varphi \wedge \text{EX } Z$
  - $\Box \nu Z. \bigwedge_i \text{EX} [ \text{E } Z \text{ U } (Z \wedge c_i) ]$
- Reachability:
  - $\Box \mu Z. I \vee \text{EY } Z$

# Outline

- ✓ Characteristic functions and BDDs
- ✓ Symbolic Model Checking
- Image/preimage computation
  - Fixpoint computation
  - Cycle detection
  - Don't care conditions

# EX: Preimage Computation

⇒  $\forall \exists y . [T(x,y) \wedge Z(y)]$  is called **preimage** computation

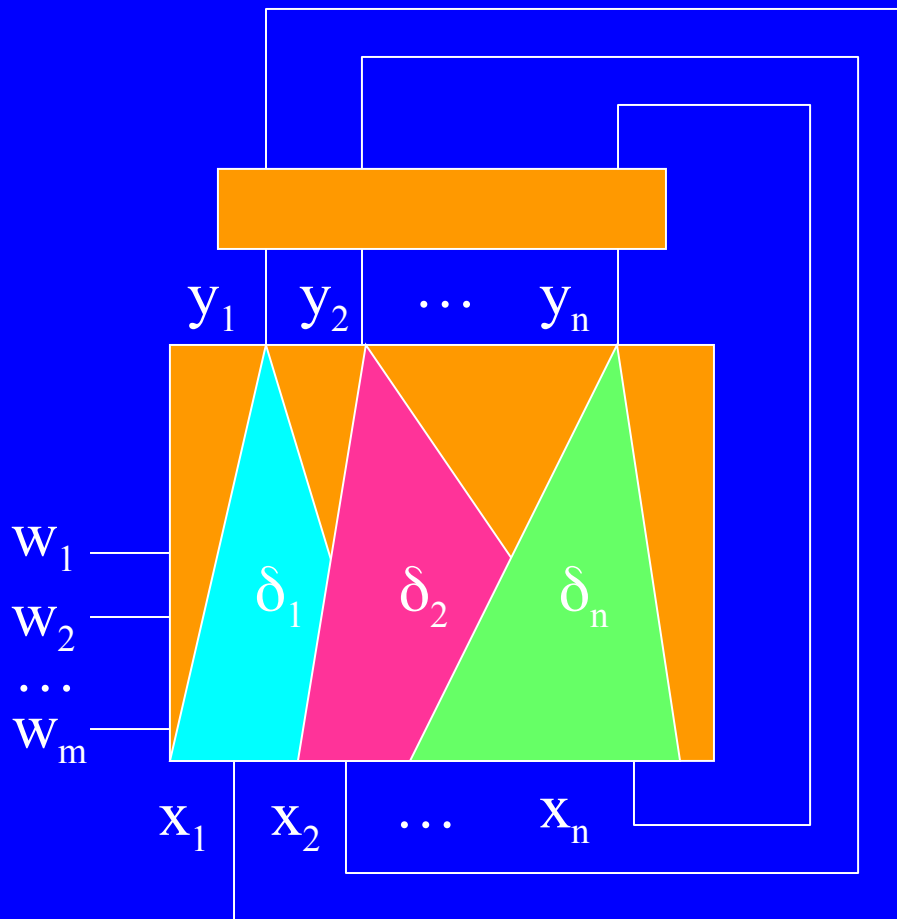
- The BDD for  $T$  is often too large
- The transition relation is kept in **partitioned** form, e.g.,
  - One term for each subcircuit/latch
  - One term for each asynchronous process

# EY: Image Computation

$\forall \exists x . [T(x,y) \wedge Z(x)]$  is called **image computation**

- It computes the set of successors to the states in  $Z(x)$
- The partitioned representation of the transition relation is also useful

# T for Synchronous Circuits



$$T(x, w, y) =$$

$$[y_1 \leftrightarrow \delta_1(x, w)]$$

$$\wedge [y_2 \leftrightarrow \delta_2(x, w)]$$

$$\dots$$

$$\wedge [y_n \leftrightarrow \delta_n(x, w)]$$

# Adding Cut-Point Variables

- Sometimes even the BDD for a single bit relation may be too large
- We can add intermediate variables that are then quantified during image/preimage computation
- We can add many intermediate variables and then let clustering get rid of those that are less useful

# Early Quantification

$$\forall \exists v_1. g(v_2) \wedge f(v_1, v_2) = g(v_2) \wedge \exists v_1. f(v_1, v_2)$$

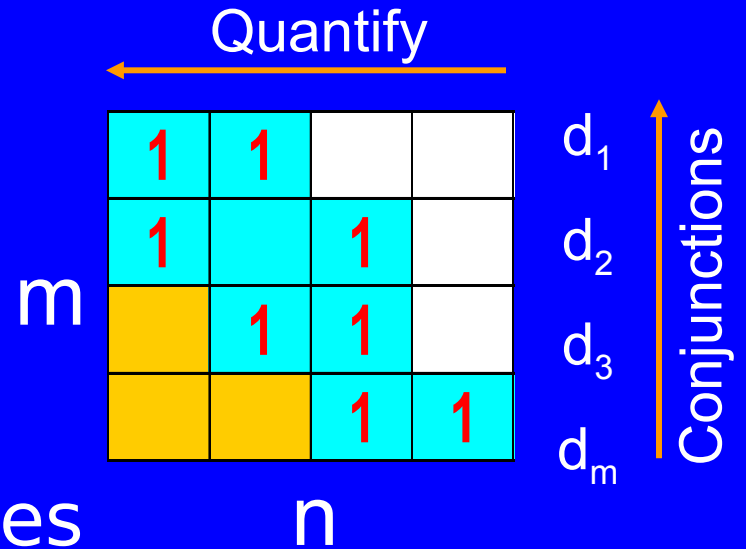
- If a variable to be existentially quantified appears in one conjunct only, it can be quantified before conjunction
- Reduction in the support of the intermediate results often translates into smaller BDDs

# Ordering and Clustering

- The parts of the transition are ordered in an attempt to heuristically reduce the sizes of the intermediate results
- Ordering tries to keep the supports of the intermediate BDDs small
  - Introduce variables as late as possible
  - Quantify variables as early as possible
- Clustering produces fewer parts so as to speed up the computation

# Dependence Matrix

- Dependence Matrix
  - $m$  : number of functions
  - $n$  : number of variables
  - $d_{ij} = 1$  :  $i$ -th function depends on  $j$ -th variable



- Average Variable Lifetimes

- $\lambda$  : total lifetime (exit)

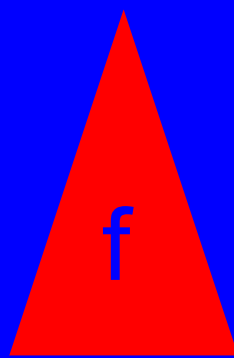
- $\alpha$  : active lifetime (entry & exit)

$$\lambda = \frac{\sum_{1 \leq j \leq n} (m - l_j + 1)}{m \bullet n} \quad \alpha = \frac{\sum_{1 \leq j \leq n} (h_j - l_j)}{m \bullet n}$$

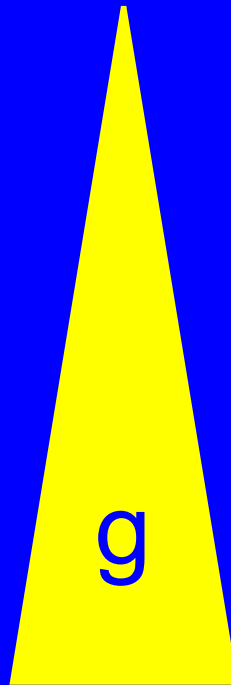
$$\lambda = (4+4+3+1) / (4 \times 4)$$
$$= 12 / 16$$

$$\alpha = (2+3+3+1) / (4 \times 4)$$
$$= 9 / 16$$

# Active Variables



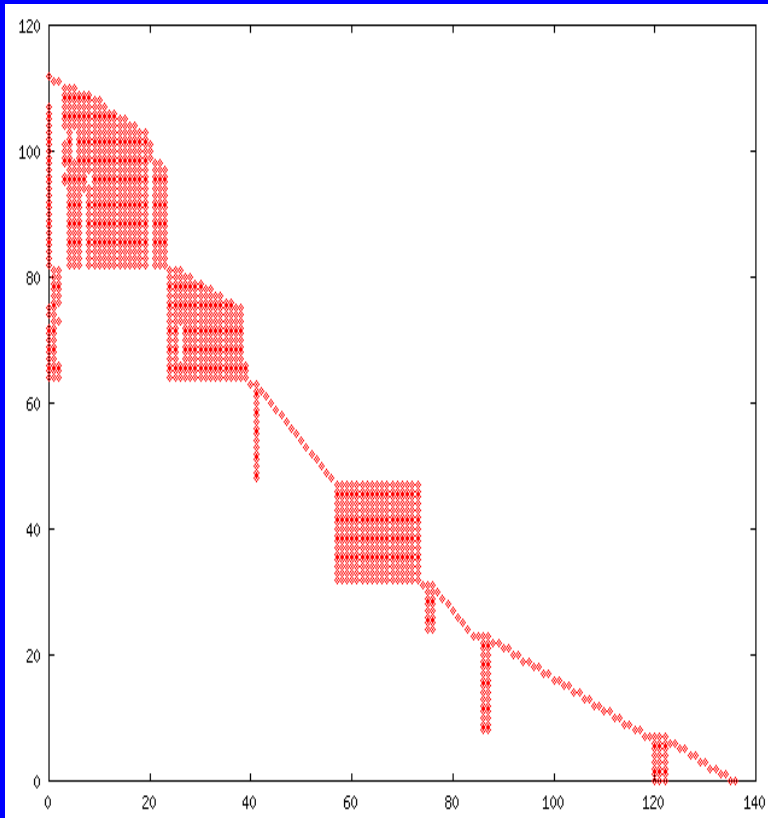
^



Recursion never  
deeper than f

- In general, there is an advantage in having some variables only in one of the operands, as opposed to having them in both operands

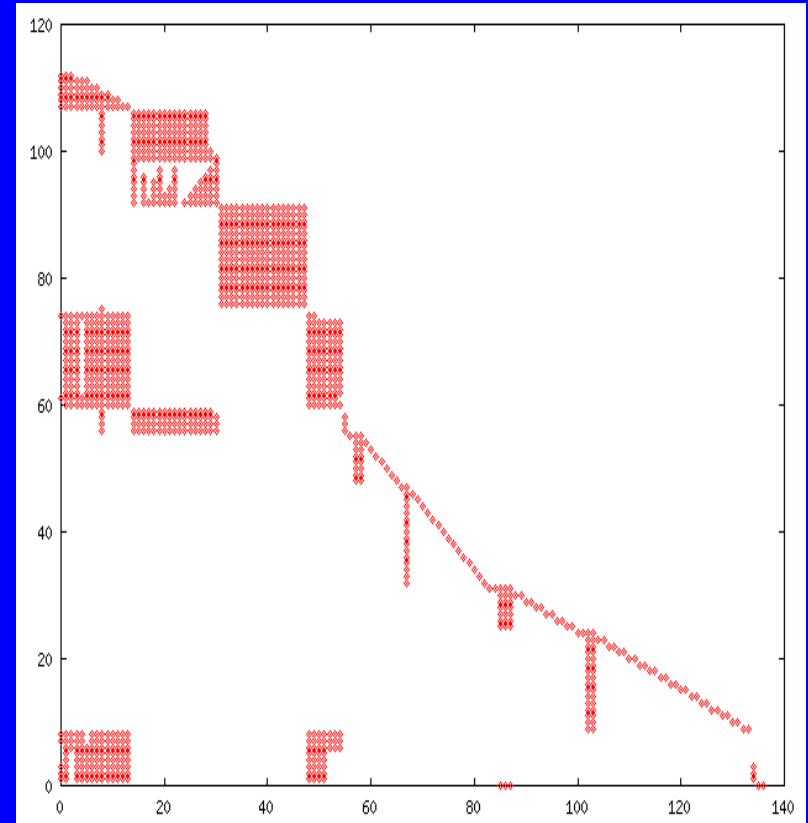
# Example (s4863)



MLP

$(\lambda = 0.38, \alpha = 0.07)$

Reachability : 2805 sec



IWLS95 [RAB+95]

$(\lambda = 0.45, \alpha = 0.20)$

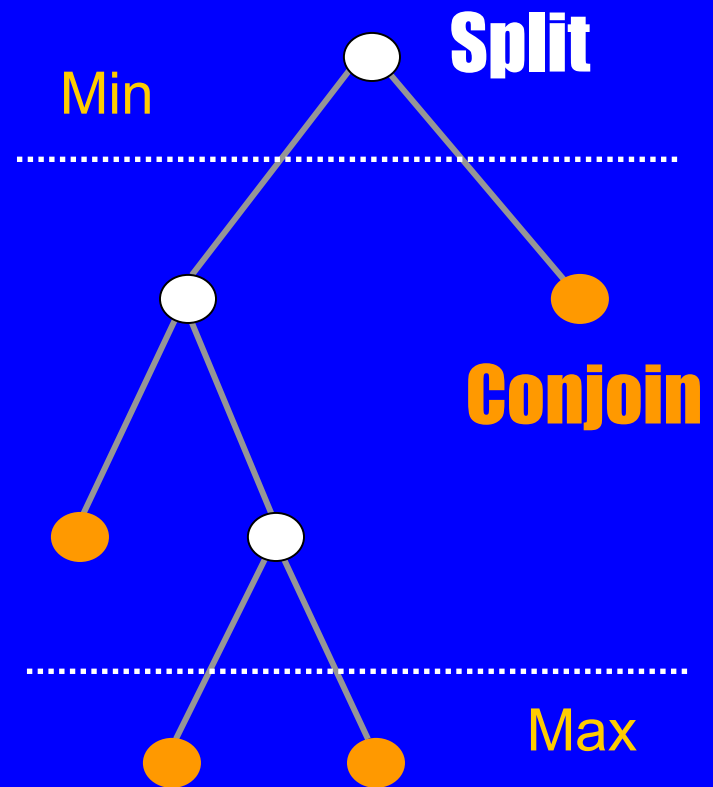
time-out

# Disjunctive Partitioning

- If the transition relation is disjunctively decomposed, quantification distributes
$$\forall \exists x . (f \vee g) \wedge h = (\exists x . f \wedge h) \vee (\exists x . g \wedge h)$$
- Disjunctive partitioning is natural for **asynchronous** systems
- It can also be applied to difficult synchronous systems by splitting on the values of a variable
- Splitting can be used to compute images and preimages without any conjunction

# Hybrid Image Computation

- At each node decide whether to split or conjoin
- If splitting, choose a variable and recur
- At each node with children disjoin their results



# Outline

- ✓ Characteristic functions and BDDs
- ✓ Symbolic Model Checking
- ✓ Image/preimage computation
- **Fixpoint computation**
  - Cycle detection
  - Don't care conditions

# Using Frontiers for $E\psi U\varphi$

```
Z =  $\varphi$ ;  
 $\zeta$  =  $\varphi$ ;
```

```
While ( $\zeta \neq \emptyset$ ) {  
  Y =  $\psi \wedge EX \zeta$ ;  
   $\zeta$  =  $Y \wedge \neg Z$ ;  
  Z =  $Z \vee \zeta$ ;  
}
```

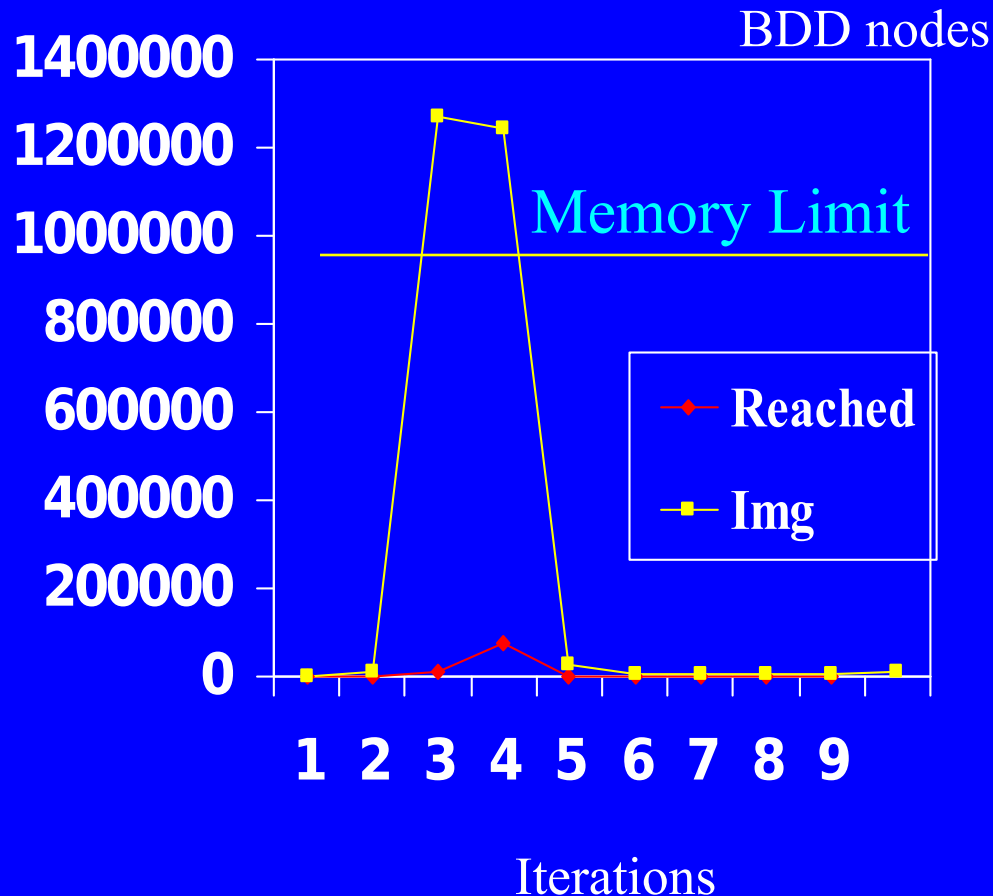
$\zeta$  = function with small  
BDD such that  
 $Y \wedge \neg Z \leq \zeta \leq Y \vee Z$



# Modified Fixpoint Computation

- Fixpoint computations are normally BFS traversals
- Problem
  - BDD size explosions in intermediate BDDs
- Cause (sometimes)
  - BFS is inflexible in controlling BDD sizes

# Reachability Analysis of S1269



# Mixed BFS/DFS

- Mixed (BFS + DFS) state search
  - Can control the size of intermediate BDDs
  - More efficient than BFS in many cases
- Techniques
  - High density traversal [RS95, RS99]
  - Partitioned traversal [CCQ96, NIJ+97]
  - Prioritized traversal [FKZ+00]

# Mixed BFS-DFS Search

- BDD Approximations [RS95, RMSS98]
  - Density of  $f$ 
$$\delta(f) = \text{minterms}(f) / \text{nodes}(f)$$
  - Redirect low-density branches to other nodes
- BDD decomposition [CCQ96, NIJ+97, FKZ+00]
  - Balanced partitions
  - Splitting on variables

# Symbolic Guided Search

- A **hint** is a (user-provided) predicate that restricts the transition relation
- Hints can be used to produce mixed BFS-DFS search
  - BDDs for fixpoint iterates are kept small
- For each hint a new fixpoint is computed using the result from the previous hint as starting point
- To guarantee convergence, the last hint leaves the transition relation unchanged

# Approximate Model Checking

Exact



$T, I \models \varphi$

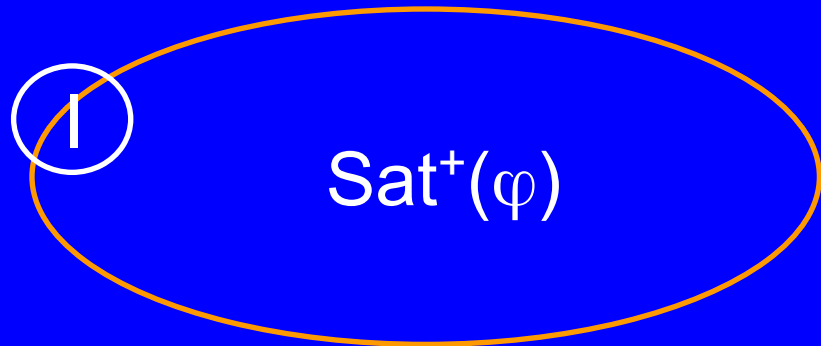


$T, I \not\models \varphi$

Approximate

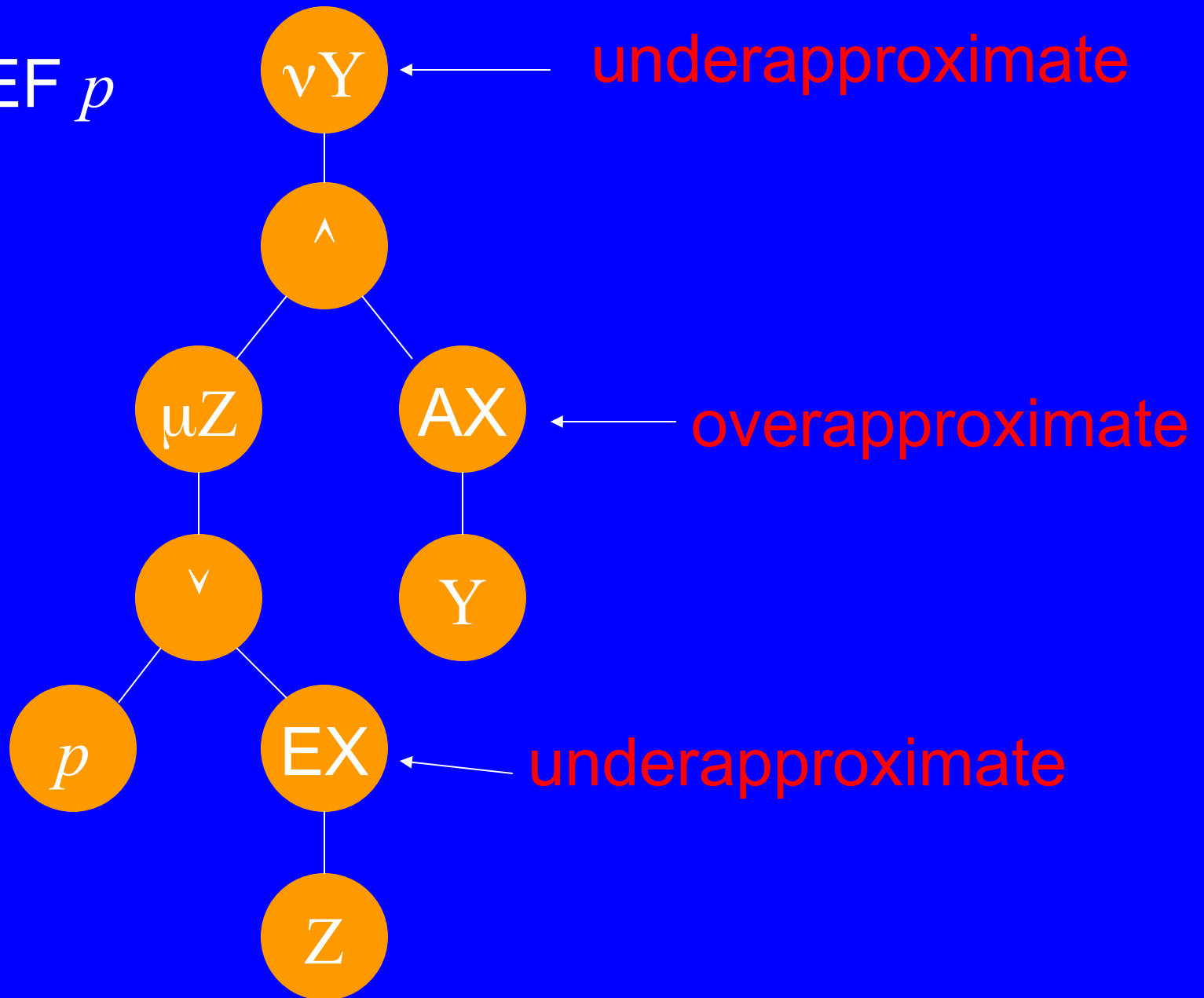


$T, I \models \varphi$



$T, I \not\models \varphi$

$AG\ EF\ p$



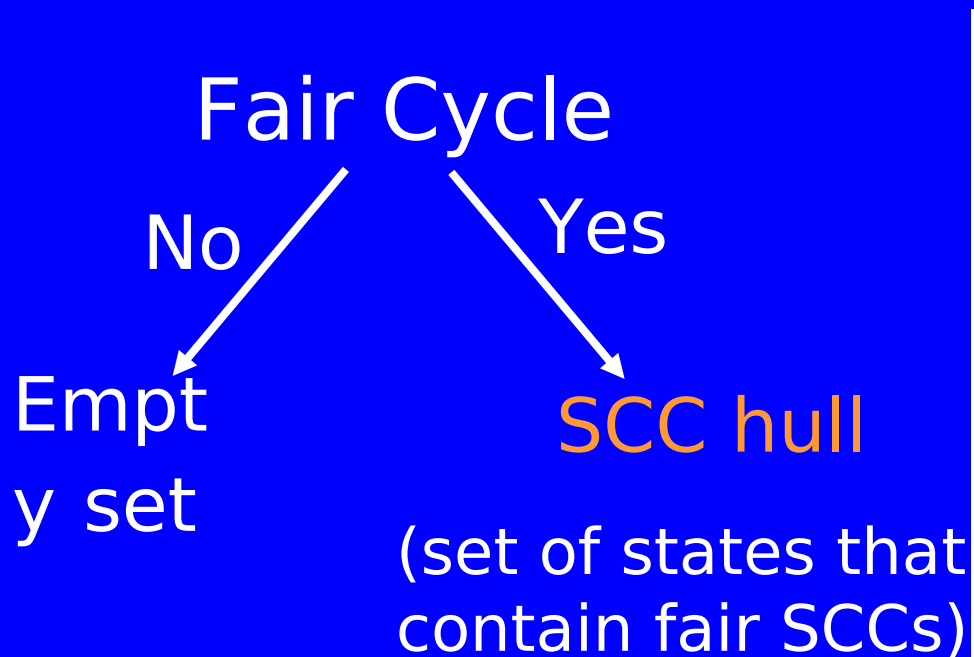
# Approximate Reachability

- Model  $\langle T, I \rangle$  is decomposed into **submodels**  $\langle T_i, I_i \rangle$  such that
  - $T \leq \bigwedge_i T_i$  and  $I \leq \bigwedge_i I_i$
  - Each submodel depends on a few variables
- Reachable states are given by iteration
  - $L_j^{-1} = I_j$
  - $L_j^i = L_j^{i-1} \vee \text{Img}(T_j, \bigwedge_k L_k^{i-1})$  if  $j = \sigma(i)$
  - $L_j^i = L_j^{i-1}$  otherwise

# Outline

- ✓ Characteristic functions and BDDs
- ✓ Symbolic Model Checking
- ✓ Image/preimage computation
- ✓ Fixpoint computation
- Cycle detection
  - Don't care conditions

# Symbolic Fair Cycle Computations



Check if each  
SCC is fair

Refinement

Symbolic  
SCC  
enumeration

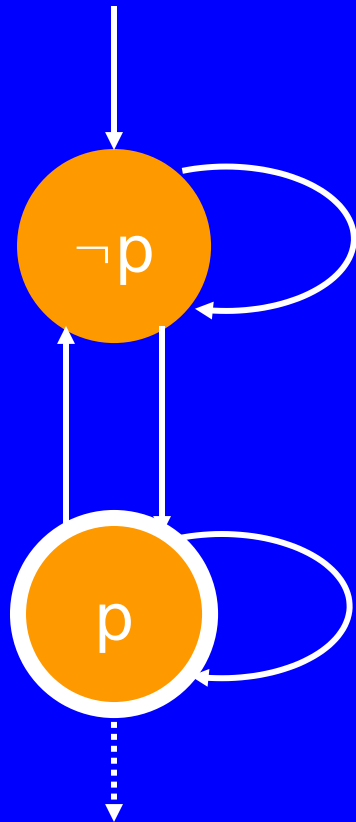
# Generic SCC Hull Algorithm (GSH)

- **SCC hull**: a set of states that contains all fair SCCs
- Generalize EL:  $\forall Z. \bigwedge_i \text{EX} [E Z U (Z \wedge c_i)]$
- Operators
  - $T_B : \text{EX}(Z), \{E(Z U Z \wedge c_i)\}$
  - $T_F : \text{EY}(Z), \{E(Z S Z \wedge c_i)\}$
- Algorithm: start with all states, at every iteration
  - choose and apply an operator from  $T_B$  or  $T_F$  (**operator schedules**)
  - converge when no change in state set under  $T_B$  **OR**  $T_F$  operators

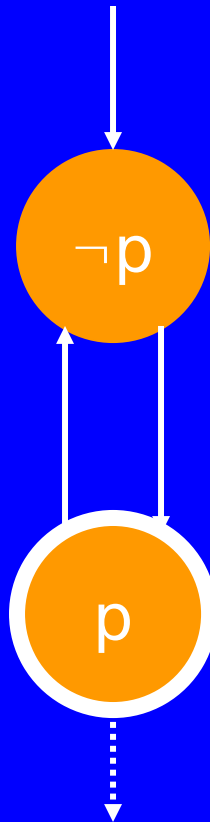
# Symbolic SCC Enumeration

- Find an SCC
  - pick a state  $v$
  - compute the SCC of  $v$  as  $(EP(v) \wedge EF(v))$
- Check if SCC is fair
- Recur on the partitions

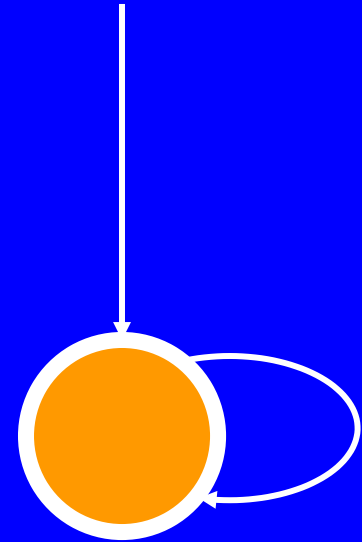
# Accepting SCC Strength



strong



weak



terminal

# Emptiness Check

- Algorithm based on property automaton
- For terminal automata: **EF fair**
  - reachability analysis
- For weak automata: **EF EG fair**
  - reachability and existence of cycle
- For strong automata: **EG<sub>fair</sub> true**
  - reachability and existence of cycle through some fair states

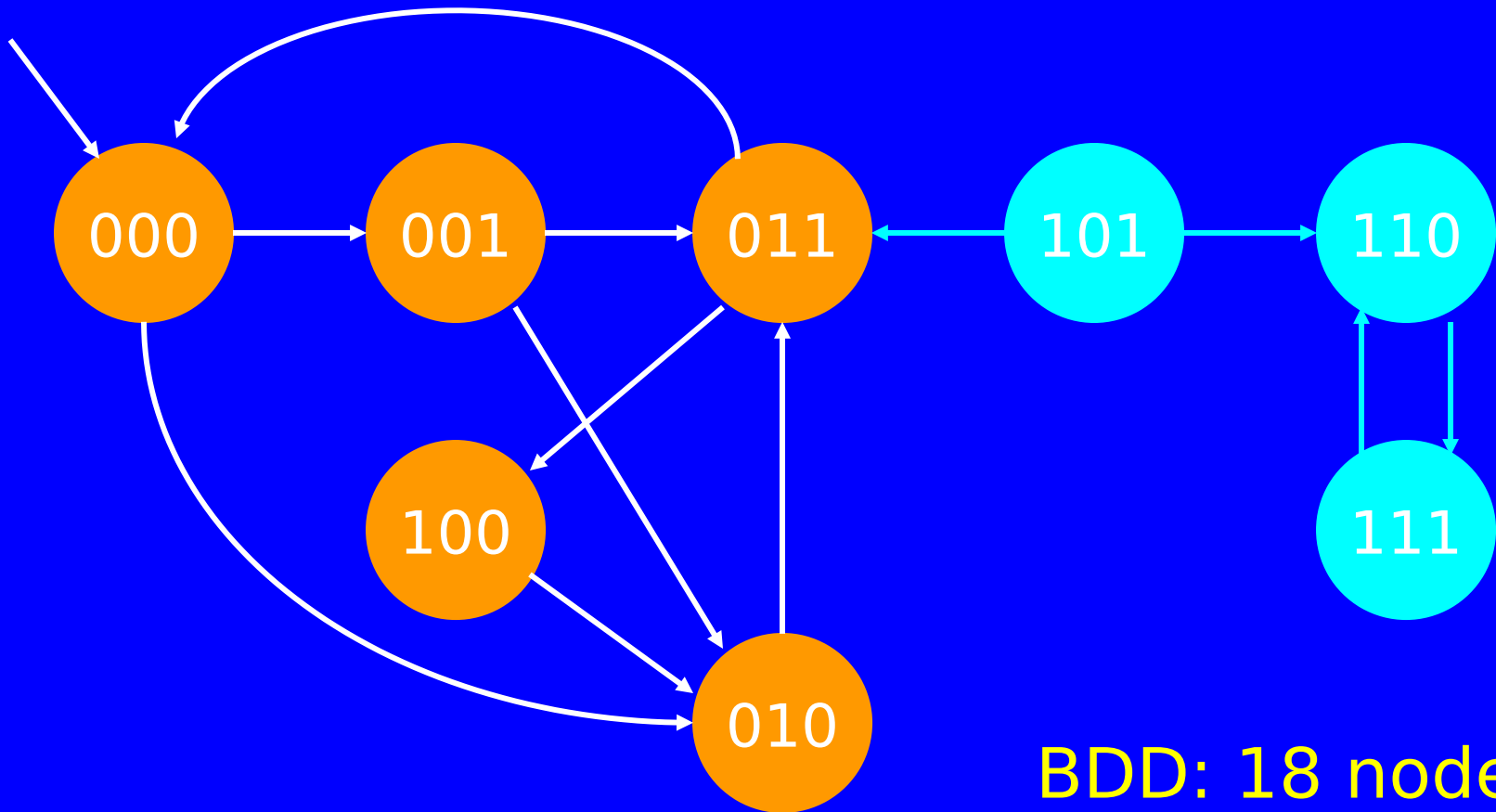
# Outline

- ✓ Characteristic functions and BDDs
- ✓ Symbolic Model Checking
- ✓ Image/preimage computation
- ✓ Fixpoint computation
- ✓ Cycle detection
- Don't care conditions

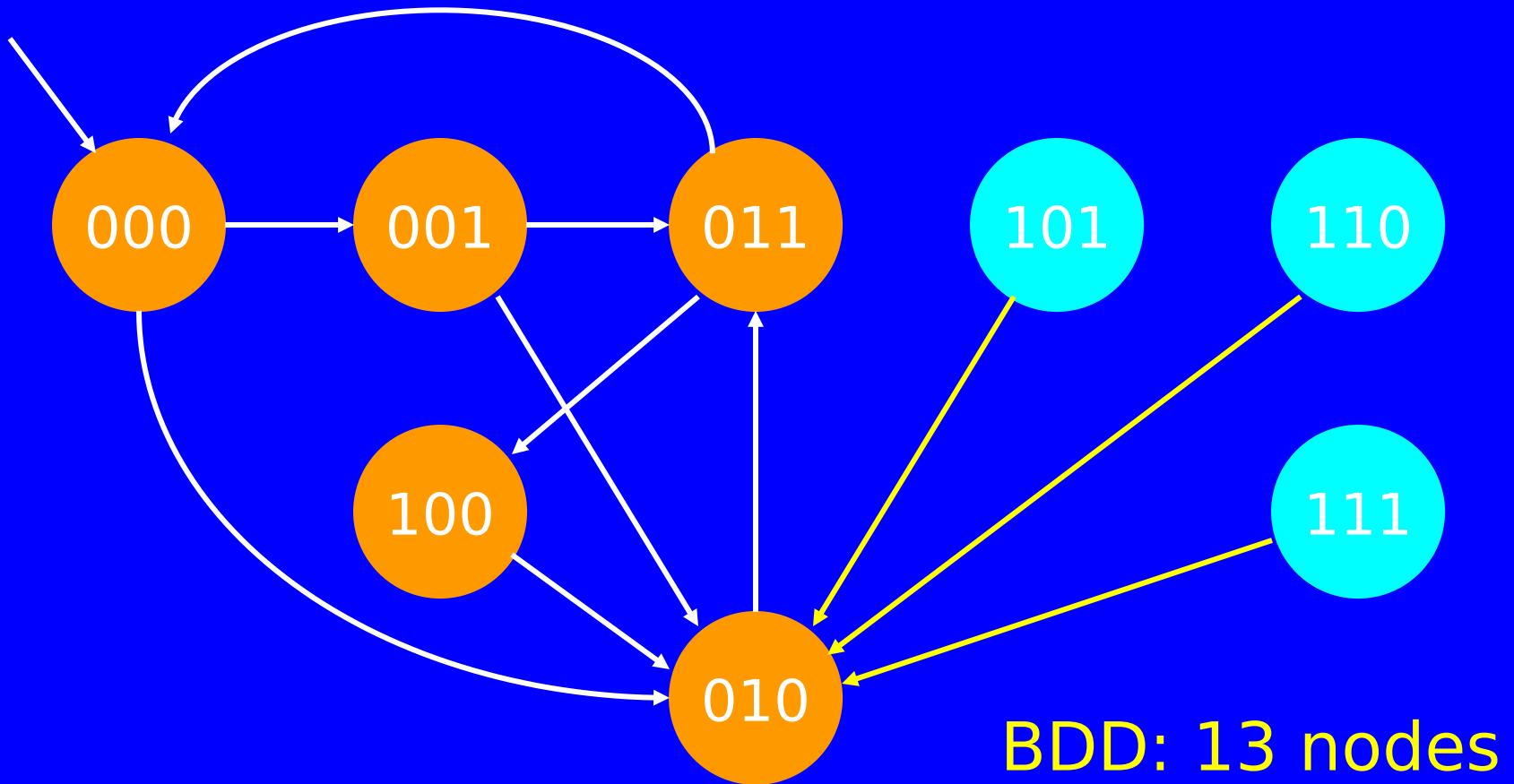
# Don't Cares

- Forward search can identify reachable states
- Backward search can be restricted to reachable states
  - Or just some unreachable states may be excluded
- The transition relation can be changed
  - Unreachable transitions can be redirected at will

# Don't Cares



# Don't Cares



# Forward Model Checking

- A subset of CTL\* can be model checked by using EY and not using EX
- Example: for a model with initial state  $s_0$ 
  - $K, s_0 \models EF\varphi$  translates into
  - $EP_{s_0} \wedge \varphi \neq \emptyset$
- Especially attractive for properties like
  - $EG \varphi$ , where  $\varphi$  does not hold in initial state
  - $E \psi U \varphi$ , where  $\psi$  holds in very few states

# Equivalent Variables

- Some state variables may have the same or complementary values on all reachable states
- Idea (TiGeR): Instantiate a BDD variable for each equivalence class of variables (initially one) and then refine the partition as reachability progresses
- Create the transition relation only in terms of the instantiated variables

# Credits

Many thanks to

- Roderick Bloem
- In-Ho Moon
- Kavita Ravi

who are the authors of some of the  
slides